

MDS5111: Python Programming

Course Project: FlashMart Retail Operations Analytics and Demand Forecasting

Corresponding Teaching Assistant: Yu Mao (118010224@link.cuhk.edu.cn)

Term 2, 2026

Project Overview

Weight: 15% of Total Grade

Deadline: April 30th 23:59

Submission: Single `your_student_id.py` file via Blackboard.

Official Runtime: Python 3.10.20 with the package versions listed in `requirements.txt`.

Important: During grading, we will only copy your submitted Python file into the official repository as `src/project.py`, and run it together with the original `src/config.py`, `src/utils.py`, and the released `data/` folder.

1 Background

You are appointed as a Junior Data Analyst at **FlashMart**, an instant retail platform operating multiple stores across several cities. The business wants a robust Python workflow that can ingest multiple business tables, audit schemas, clean operational data, generate business KPIs, build time-series features, train a basic demand-forecasting model, and produce management-facing visualizations.

Your task is to complete the starter script so that the full pipeline runs end-to-end and produces the required CSV outputs and figures.

2 Repository Structure

The released repository has the following structure:

- `data/`: released CSV tables
- `figures/references/`: reference images for the two required figures
- `outputs/`: generated CSV outputs
- `src/config.py`: project constants (do not modify)
- `src/utils.py`: utility functions (do not modify)
- `src/project.py`: the **only** file you should implement
- `setup_and_run.py`, `setup_and_run.sh`, `setup_and_run.ps1`: setup-and-run scripts
- `requirements.txt`, `runtime.txt`: official environment specification

3 Environment Setup

Official recommendation

Use the environment specification released in the repository. You **do not need to replace your system Python**. Instead, create an isolated environment for this assignment. We strongly recommend using the provided setup scripts.

Recommended one-command setup

Windows

```
py -3.10 setup_and_run.py
```

or

```
.\setup_and_run.ps1
```

macOS / Linux

```
python3.10 setup_and_run.py
```

or

```
bash setup_and_run.sh
```

These scripts will:

1. create a local virtual environment named `.venv`,
2. install the required packages from `requirements.txt`,
3. set the official runtime environment variables,
4. run:

```
python src/project.py --data_dir data --output_dir outputs --figure_dir  
figures
```

4 The Dataset

You will work with the six CSV files in `data/`:

- `orders_train.csv`: daily training data at the `date × store_id × category` level. Key fields include `orders`, `gmvt`, `stockout_minutes`, `avg_delivery_time`, and `refund_rate`.
- `orders_test.csv`: future rows with the same business granularity but without the target variable `orders`.
- `store_info.csv`: store metadata, including `city`, `city_tier`, `business_district_type`, `store_size`, and `opened_days`.
- `calendar.csv`: calendar attributes, including `day_of_week`, `month`, `week_of_month`, `is_weekend`, `is_month_start`, `is_month_end`, `is_payday_window`, and `is_holiday`.
- `promotion.csv`: promotion descriptors at the `date × store_id × category` level, including `campaign_id`, `promo_type`, `promo_depth`, and `coupon_flag`.
- `city_external.csv`: city-level external signals such as `rain_level`, `temperature_bin`, `traffic_index`, and `event_flag`.

5 Implementation Requirements

You must complete the starter template in `src/project.py`. There are 16 tasks in total, which are also listed in the `src/project.py` using comments.

General Rules

- You should only modify `src/project.py`.
- Do **not** modify `src/config.py` or `src/utils.py`.
- Use only the released CSV files in `data/`. Do **not** regenerate or replace the data.
- Do **not** introduce extra randomness, shuffling, sampling, or manual re-ordering unless a task explicitly asks for it.
- Keep all required function names, variable names, output file names, and output column names unchanged.

Section 1: Data Loading and Schema Audit

1. `load_raw_data(data_dir)`: Read all six input tables using the exact file names defined in `config.py`. Any table containing a `date` column must parse that column as `datetime`.
2. `build_schema_summary(raw)`: Build a schema summary table with the exact fields:
 - `table_name`
 - `n_rows`
 - `n_cols`
 - `n_missing_total`
 - `date_min`
 - `date_max`

Section 2: Merge and Cleaning

3. `merge_master(order_df, stores, calendar, promo, external)`: Merge the training orders table with the auxiliary tables using left joins, in the exact order required by the starter code.
4. `clean_master(df)`: Fill missing values for the required zero-fill columns and record the number of filled entries in the cleaning log.
5. Repair invalid `discount_rate` values by clipping them into $[0, 1]$.
6. Repair invalid `stockout_minutes` values by clipping them from below at 0.
7. Repair invalid `avg_delivery_time` values by replacing values ≤ 0 with `NaN`.

Section 3: Business Analysis

8. Build a category-level volatility summary:
 - group by `category`,
 - compute the mean and standard deviation of `orders`,
 - rename them to `orders_mean` and `orders_std`,
 - compute `orders_cv`.
9. Compute promotion lift for each (`store_id`, `category`) pair using the provided `promo_lift_func`.
10. Create a city-by-category pivot table of average orders.
11. Build a store-level alert table:
 - compute mean `orders`, `stockout_minutes`, `refund_rate`, and `gmv`,
 - add `high_orders_high_stockout_flag`, defined as 1 only when both `avg_orders` and `avg_stockout_minutes` are strictly above their 75th percentiles.

Section 4: Time-Series Feature Engineering

12. In `add_lag_rolling_features(df)`, create the grouped time-series features:
 - `rolling_mean_7`
 - `rolling_std_7`
 - `rolling_mean_28`
 - `promo_last_7d_count`
 - `stockout_last_7d_mean`

All rolling features must be calculated within each (`store_id`, `category`) group, and `shift(1)` must be applied before every rolling calculation to avoid leakage.

Section 5: Modeling

13. `prepare_model_matrix(train_df, valid_df, feature_cols)`:
 - select `feature_cols` + `CATEGORICAL_COLS`,
 - one-hot encode only the categorical columns with `drop_first=True`,
 - align validation columns to training columns using left alignment on `axis=1`,
 - return `X_train`, `X_valid`, `y_train`, `y_valid`.
14. `train_formal_model(train_df, valid_df)`:
 - use `FEATURE_COLS_BASE` + `FEATURE_COLS_TS`,
 - remove rows with missing values in required features or target,
 - fit a default `LinearRegression()`,
 - clip predictions from below at 0,
 - build the exact one-row metrics table with columns: `model_name`, `mae`, `rmse`, `mape`, `n_valid_rows`.

Section 6: Visualization

15. `plot_daily_orders_overview(daily_df)`: create an executive overview figure with:

- one line for total daily orders,
- one line for a 7-day rolling mean,
- highlighted high-promotion periods,
- one inset showing the final 28 days,
- at least two annotations,
- one legend.

Use colors to distinguish the key visual elements. The plot must remain readable: legend, inset, annotation text, and highlighted regions should not cover the main patterns or overlap badly. Your final output should be broadly similar to `figures/references/fig_a.png`.

16. `plot_validation_diagnostics(valid_df)`: create a validation diagnostics figure with:

- a scatter plot of actual vs predicted values,
- a 45-degree reference line,
- a residual inset inside the same figure,
- annotations for the top 3 largest errors.

Use colors to distinguish the key visual elements. The residual inset should be a residual-distribution view rather than an unrelated summary plot. The figure must remain readable and reasonably well arranged: labels, annotations, points, inset, and legend (if used) should not obviously overlap in a way that harms readability. Your final output should be broadly similar to `figures/references/fig_b.png`.

6 Expected Outputs

When your script runs successfully, it should generate the following files:

CSV outputs in `outputs/`

- `01_schema_summary.csv`
- `02_cleaning_log.csv`
- `02_master_sample.csv`
- `03_city_kpi.csv`
- `03_category_stats.csv`
- `03_promo_lift.csv`
- `03_city_category_pivot.csv`
- `03_store_alerts.csv`
- `04_feature_dictionary.csv`

- 04_feature_preview.csv
- 04_train_valid_split_summary.csv
- 05_model_metrics.csv
- 05_ablation.csv (*scaffold output; not graded*)
- 06_error_by_city.csv
- 06_error_by_category.csv
- 06_error_by_condition.csv
- 06_top10_errors.csv

Figure outputs in figures/

- fig_a.png
- fig_b.png

7 Grading Rubrics (Total: 100 Points)

The grading consists of **88 points of automatic grading** and **12 points of manual figure review**.

Category	Weight	Criteria
Section 1: Schema summary	4 pts	Correct loading and schema audit output.
Section 2: Cleaning outputs	12 pts	Correct cleaning log and cleaned master sample.
Section 3: KPI and group analysis	26 pts	Correct category statistics, promotion lift, pivot table, and store alerts.
Section 4: Feature engineering	15 pts	Correct feature dictionary, feature preview, and train/validation split summary.
Section 5: Modeling	15 pts	Correct baseline and formal model metrics.
Section 6: Error analysis	16 pts	Correct segment-level error summaries and top-10 error table.
Figure A	6 pts	Contains all required elements, uses distinguishable colors, and remains readable without harmful overlap.
Figure B	6 pts	Contains all required elements, includes a residual inset, uses distinguishable colors, and remains readable without harmful overlap.

Important Notes

- **Submission file naming:** rename your submission file to exactly your `9-digit_student_id.py` (e.g., `123456789.py`).
- **Interface integrity:** do not rename, remove, or modify the provided function names, expected output file names, or required column names. The grading pipeline relies on these identifiers.
- **Code scope:** only the content of your submitted Python file will be graded. We will use the official `config.py`, `utils.py`, and released data files.
- **Late submission:** penalty policy follows the course policy.
- **Zero tolerance:** scripts that fail to import or contain syntax errors will receive a score of 0.

8 Final Advice

Before submission:

1. test your code in the official repository structure,
2. use the provided setup scripts to reproduce the official environment,
3. verify that all required CSV files are written to `outputs/`,
4. verify that `fig_a.png` and `fig_b.png` are generated and visually readable,
5. compare your figures with the reference images in `figures/references/`.